Shoulder Rotation Change Report

Haemophilia physiotherapy project

12 October 2026 | Branch feature/shoulder-rotation-model

Status Local source changes only

The code now follows the requested bilateral repetition definition and includes fixes to tracking, camera overlay mapping, form feedback and training checks. The installed phone app and the backend it connects to have not been updated. No commit, push, merge, APK installation or hosted deployment was performed.

Verification passed during implementation. The revised model still needs reviewed training sequences and evaluation, and physical phone testing remains outstanding.

Repetition definition

Centre -> one extreme -> centre -> opposite extreme

Count once when both hands reach the opposite extreme. Either side can come first. After counting, both hands must return to centre before another repetition can begin.

- Centre -> left -> centre -> right counts once at right.
- Centre -> right -> centre -> left counts once at left.
- Centre -> one side -> centre is only a partial repetition.
- Starting at an extreme requires a centre visit before a valid sequence begins.
- The return to centre after a count resets the detector without adding a count.
- Tracking loss discards unfinished movement; completed counts survive.
- A completed repetition can have Incorrect form and still count.

Pipeline and API compatibility

The MediaPipe -> biomechanical features -> PyTorch LSTM -> repetition assessment pipeline remains in FastAPI and Flutter. The LSTM retains 10 inputs, 128 time steps, two layers, 64 hidden units and two classes: 0 = Correct and 1 = Incorrect. Four additional observed features support counting and form rules without changing the LSTM input size.

Existing API fields are retained. Responses add tracking status, frame identifiers, image size, debug features and a model retraining flag.

Implemented fixes

Counting and calibration

The previous live detector counted each half-cycle; CSV tooling used a different cycle definition. Both now use the shared bilateral state machine. Counting uses both hands' horizontal displacement instead of averaging depth-derived forearm angles, which could cancel movement.

Calibration first requires geometric neutral from the wand and elbow midpoint, then stores separate grip offsets. At 20 Hz, ten neutral samples calibrate the detector. Extremes require three samples; centre crossings require one. Incomplete movements expire after 25 seconds. These thresholds need broader validation.

Camera overlay and frame handling

Preview and skeleton share one aspect-ratio rectangle. Portrait capture is locked, mirroring is applied once, and tests cover coordinate rotation and crop mapping. Coordinates are no longer clamped to invented edge positions.

Camera bytes are copied before asynchronous conversion. One request is in flight; responses echo frame IDs, timestamps and image size. The client rejects obsolete IDs. Network latency can still delay the pose overlay.

Tracking and form feedback

Missing poses, low visibility, invalid coordinates and long timestamp gaps discard the incomplete repetition. Flutter uses current visibility, clears missing poses and expires stale skeletons after 500 ms.

Form rules check elbow angle, upper-arm distance from the torso axis, hand coordination, torso tilt and twisting. Sustained errors produce specific feedback. Upper-arm length is no longer treated as elbow drift.

Performance and model output

Form score is separate from uncalibrated model probabilities. Duration uses sampled time, speed uses mean absolute angular motion, and smoothness uses acceleration. The app distinguishes form score, forearm ROM and model estimate. History labels the correct-repetition percentage Correct Rep Rate.

LSTM and training safeguards

The backend default was a tabular classifier. It now defaults to the existing LSTM and checks checkpoint feature order, sequence length and normalization metadata. The old checkpoint is flagged as needing retraining for the new repetition definition.

Training rejects old extraction contracts, duplicate sequences and videos assigned across subjects. Subject splits, training-only class weights and validation-only checkpoint selection remain. Evaluation adds per-class precision, recall and F1, macro F1 and Incorrect recall.

Verification and model results

Check	Result
Python regression suite	34 tests passed
Focused Flutter suite	5 tests passed
Final Flutter analysis	No issues
Git whitespace check	Passed
A_correct_01 replay	8 candidates from 739 samples
A_incorrect_01 replay	9 candidates from 728 samples
All 17 candidate arrays	Finite 128 x 10 arrays; pending review

Tests cover direction, initial extreme, partial movement, jitter, pauses, reset, repeated repetitions, tracking loss, bad form, timing, feature calculations, API flow, coordinate mapping and synthetic training. Recorded counts are candidates, not measured detector precision or recall. Independent human counts and boundary review are still required.

Existing LSTM baseline

The original LSTM was evaluated on its frozen run04 sequences: 25 test samples from subject E. These results belong to the former half-cycle definition and do not establish accuracy of the revised pipeline.

Metric	Result
Accuracy	84.00%
Balanced accuracy	83.65%
Macro F1	83.77%

Class	Precision	Recall	F1	Samples
Correct	80.00%	92.31%	85.71%	13
Incorrect	90.00%	75.00%	81.82%	12

Confusion matrix with actual rows and predicted columns

Actual class	Predicted Correct	Predicted Incorrect
Correct	12	1
Incorrect	3	9

The old dataset has 263 accepted sequences across five IDs: 177 Correct and 86 Incorrect. New boundaries and labels need review; no production retraining was performed. Use new subjects for the final test because existing subjects were inspected in earlier experiments.

Changed files

Paths below are relative to the repository. Existing unrelated files were preserved.

Backend

- `backend/app/api/routes/live.py`
- `backend/app/api/routes/exercises.py`
- `backend/app/services/model_registry.py`

Shared features counting and assessment

- `src/exercises/shoulder_rotation.py`
- `src/exercises/shoulder_rotation_assessment.py`
- `src/features/shoulder_rotation_features.py`
- `src/feedback/rotation_cycle_detector.py`
- `src/feedback/shoulder_rotation_rep_counter.py`
- `src/models/shoulder_rotation_loader.py`
- `scripts/shoulder_rotation_pipeline.py`

Flutter camera display and tutorial

- `haemophilia_app/lib/screens/assessment/live_assessment_screen.dart`
- `haemophilia_app/lib/utils/pose_coordinates.dart`
- `haemophilia_app/lib/screens/patient/patient_history.dart`
- `haemophilia_app/lib/widgets/exercise_demo/exercise_demo_model.dart`
- `haemophilia_app/lib/widgets/exercise_demo/shoulder_rotation_painter.dart`

Tests

- `tests/test_rotation_cycle_detector.py`
- `tests/test_shoulder_rotation_timing.py`
- `tests/test_shoulder_rotation_backend.py`
- `tests/test_shoulder_rotation_features.py`
- `tests/test_shoulder_rotation_pipeline.py`
- `haemophilia_app/test/pose_coordinates_test.dart`

Documentation

- `docs/shoulder_rotation_change_report.md`
- `docs/shoulder_rotation_app_integration.md`
- `docs/shoulder_rotation_runbook.md`
- `docs/shoulder_rotation_accuracy.md`

The open Haemophilia/src/inference/live_camera.py and older live_camera.py copies implement assisted shoulder flexion. Rotation changes are in the shared engine used by FastAPI and Flutter.

Deployment and remaining work

The installed app will receive all changes only after the backend it connects to is updated and the Flutter app is rebuilt and installed.

1. Deploy or restart the updated backend with the selected LSTM and its config.json.
2. Rebuild and install the updated Flutter app to receive overlay and UI changes.
3. Test physical phone alignment, front and back cameras, tracking loss and latency.
4. Review regenerated boundaries and labels, then retrain and evaluate the LSTM.

Local backend test commands

Run in PowerShell from the repository root. Leave this terminal running.

```
.\.venv-shoulder\Scripts\python.exe -m uvicorn `
  backend.app.main:app --host 0.0.0.0 --port 8000
```

Android phone test commands

Connect the phone by USB with USB debugging enabled. Run in another terminal and replace YOUR_DEVICE_ID with the phone ID shown by flutter devices.

```
cd haemophilia_app
adb reverse tcp:8000 tcp:8000
flutter devices
flutter run -d YOUR_DEVICE_ID `
  --dart-define=BACKEND_URL=http://127.0.0.1:8000 `
  --dart-define=POSE_DEBUG=true
```

For hosted testing, use the actual hosted backend URL in BACKEND_URL and deploy the updated backend there. Local edits and a GitHub push alone do not update an installed app.

Dataset review and limitations

Review 17 candidates in processed_data/shoulder_rotation/exact_contract03/annotations.csv; traces, diagnostics and arrays are beside it. Extraction, splitting, training and evaluation commands are in docs/shoulder_rotation_runbook.md.

Single-camera forearm angles and depth approximate visible motion, with limited anatomical accuracy. Form thresholds and scores need validation. Camera conversion requires YUV420; BGRA-only streams need another conversion path.

Assisted elbow loads. Other exercise verification is blocked by the pre-existing missing models/assisted_shoulder_lstm.pth and models/elbow_lstm.pth. Restore their authoritative checkpoints.

Saved evaluation evidence

```
reports/shoulder_rotation_exact/baseline/test_metrics.json
reports/shoulder_rotation_exact/baseline/test_predictions.csv
reports/shoulder_rotation_exact/dataset_audit.json
processed_data/shoulder_rotation/exact_contract03/detection_diagnostics.json
```